

Starling 2 메인 가이드

ModalAI Starling 2 (VOXL 2) 실사용 가이드

2026-09-02

Table of Contents

시작하기	4
우리 드론 정보	4
SKU, 버전, 접속 정보	4
WIFI	5
첫 부팅	5
네트워크 연결	5
SoftAP 모드	5
Station 모드	6
모드 전환	6
SSH 접속	7
비밀번호: oelinux123	7
개발 환경	7
VSCode 설치 (v1.85.2)	7
Remote-SSH 연결	7
파일 전송 (SCP)	8
nano 편집기	8

조종기 사용	9
스위치 배치	9
비행 모드	10
시동/종료 절차	10
수동 비행 팁	11
자율 비행	11
MAVSDK 서버 실행.....	11
비행 코드 실행.....	13
안전 절차	13
로그 확인	14
웹 대시보드 (Portal)	14
접속	14
주요 페이지	14
활용 팁	15
자주 하는 작업.....	15
발열 확인	15
CPU/프로세스 확인.....	16
로그 다운로드.....	16
파라미터 백업.....	16
트러블슈팅.....	17

시동 안 걸릴 때	17
EKF 관련 에러	18
VSCoDe 연결 실패.....	18
발열 문제	19
배터리 관련	19
안전 수칙	20
비행 전 체크리스트	20
비상 대응	20
배터리 관리	21
패치노트	21
nano 설치 완료	21
파이썬 3.6.9 환경.....	21
mavsdk 0.12.0	21
vscode 설치 1.85.2.....	22
참고 자료	22
기술 문서 링크.....	22
공식 docs 링크	22

이 문서는 ModalAI 공식 문서(docs.modalai.com), ModalAI 포럼, PX4 공식 문서를 기준으로 작성했으며, “우리 드론”에 대한 항목은 연구실에서 실제로 확인한 값으로 채워 넣도록 [확인 필요] 표시를 남겼습니다. 공식 문서에서 확인되지 않은 항목은 (미확인)으로 표시했습니다.

시작하기

우리 드론 정보

SKU, 버전, 접속 정보

항목	값	확인 방법
기종	ModalAI Starling 2 (플랫폼 코드 D0014-4)	—
컴퓨터 보드	VOXL 2 (M0054, Qualcomm QRB5165)	vox1-version
SKU	[확인 필요] 예: D0014-4-C26-M22-T7	cat /data/modalai/sku.txt
카메라 구성	C26 = hires(IMX412) + ToF + tracking 2 개(AR0144 전방/하방) / C27 = tracking 3 개	SKU 의 c 코드
RC 수신기	[확인 필요] T7 = ELRS 915 MHz (M0184) / T9 = Ghost 2.4 GHz	SKU 의 t 코드
시스템 이미지 / vox1-suite 버전	[확인 필요]	vox1-version
OS	Ubuntu 18.04 (aarch64), 커널 4.19	cat /etc/os-release
Python	3.6.9	python3 --version
SSH 계정	root / oelinux123	—
SoftAP 기본 IP	192.168.8.1	—
Station 모드 IP (연구실 WiFi)	192.168.45.157 (DHCP 라 바뀔 수 있음)	vox1-my-ip

SKU 문자열 형식은 D0014-4-CXX-M22-TX 이며, CXX 가 카메라 구성, TX 가 RC 수신기 종류를 의미합니다. ModalAI 가 Starling 2 에 권장하는 SDK 는 **1.6.4 이상**(비행 검증 완료)이고, 2026 년 7 월 기준 최신 안정판은 **SDK 1.6.6** 입니다.

WIFI

- 공장 출하 SoftAP: **SSID VOXL-2969670613 / PW 1234567890** (5 GHz AP, VOXL IP 192.168.8.1)
- 우리가 만든 SoftAP: **SSID VOXL-m0054 / PW 1234567890** (2.4 GHz)
- 연구실 Station 모드: AgtechLab 네트워크에 접속 (아래 “네트워크 연결” 참고)

첫 부팅

1. 프로펠러 장착 여부와 배터리 커넥터(XT30)를 확인합니다. Starling 2 배터리는 **2S Li-Ion 18650 (Sony VTC6 3000 mAh 또는 4000 mAh 순정팩)** 이며 LiPo 가 아닙니다. 3S 배터리는 모터가 2S 사양이므로 사용하면 안 됩니다.
2. 배터리를 연결하면 ESC 가 짧은 부팅음을 내고 VOXL 2 가 부팅됩니다(부팅 시간은 공식 문서에 명시되지 않음, 경험상 약 1 분 내외 [확인 필요]).
3. 벤치 작업 시에는 벽 전원(PSU + XT60→XT30 어댑터)을 쓸 수 있지만, **벽 전원으로는 절대 시동(arm)하지 않습니다.**
4. PC 에서 WiFi 목록에 VOXL-xxxx 가 보이면 정상 부팅입니다. 처음에는 SoftAP 로 붙어 `ssh root@192.168.8.1` 로 들어갑니다.
5. 전원을 끌 때는 그냥 배터리를 뽑아도 되지만, 파일을 수정했다면 먼저 `sync` 를 실행합니다. **SDK 플래싱 중에는 절대 전원을 끄지 않습니다**(보드가 벽돌이 될 수 있음).
6. USB-C 로 연결하면 `adb shell` 로도 접근 가능합니다(USB-C ↔ USB-A 케이블 권장, C-to-C 는 일부 PC 에서 불안정).

네트워크 연결

SoftAP 모드

- 드론이 자체 AP 가 되어 PC 가 드론에 접속하는 방식. **비행 시 기본 모드**로 사용합니다(외부 공유기 의존 없음, 지연 적음).

- 우리 설정 명령:

```
vox1-wifi softap VOXL-m0054 1234567890      # 2.4 GHz AP (채널 6)
vox1-wifi softap5 VOXL-m0054 1234567890     # 5 GHz AP (채널 149)가 필요할 때
```

- 비밀번호를 생략하면 기본값 1234567890 이 적용됩니다.
- SoftAP 에서 VOXL 의 IP 는 항상 192.168.8.1, 접속한 PC 는 192.168.8.10 부터 할당됩니다.
- 일부 노트북은 5 GHz AP 를 보지 못하므로 2.4 GHz(softap)로 만든 것입니다.

Station 모드

- 드론이 기존 WiFi(연구실 공유기)에 클라이언트로 접속하는 방식. **인터넷이 필요할 때**(apt/pip 설치, 라이브러리 다운로드) 사용합니다.

```
vox1-wifi station AgtechLab dormxpzmdusrntlf2025!
vox1-wifi station <WiFi 이름> <비밀번호>
```

- 접속 후 IP 확인: 드론에서 vox1-my-ip, 또는 USB 로 adb shell vox1-my-ip, 또는 공유기 DHCP 목록.
- 현재 연구실 IP: 192.168.45.157 (DHCP 갱신 시 바뀔 수 있으니 안 되면 위 방법으로 재확인).

모드 전환

```
vox1-wifi getmode                # 현재 모드 확인
vox1-wifi softap <ssid> [pw]     # → SoftAP(2.4 GHz)
vox1-wifi softap5 <ssid> [pw]   # → SoftAP(5 GHz)
vox1-wifi station <ssid> <pw>   # → Station
vox1-wifi factory               # 공장 초기화: 5 GHz AP VOXL-<serial> / 1234567890
reboot                          # 모드 변경 후 반드시 재부팅
```

- 모드 변경 후에는 **재부팅이 필요**하며, 설정은 재부팅 후에도 유지됩니다.
- 재부팅 후 기존 SSH 세션은 끊어지므로 새 IP 로 다시 접속합니다. SSH 가 안 되면 USB 로 adb shell → vox1-wifi getmode 로 상태를 확인합니다.

SSH 접속

```
ssh root@192.168.45.157      # Station 모드 (연구실 WiFi)
ssh root@192.168.8.1        # SoftAP 모드
```

비밀번호: oelinux123

- 기본 계정은 root, 비밀번호는 oelinux123 입니다.
- 매번 비밀번호 입력이 번거로우면 PC 에서 ssh-copy-id root@<ip>로 공개키를 등록합니다.

개발 환경

VSCode 설치 (v1.85.2)

- VOXL 2 는 Ubuntu 18.04(glibc 2.27)입니다. VS Code **1.86 부터** glibc ≥ 2.28 을 요구하고, **1.99(2025-03)부터는 Remote 서버 자체가 18.04 에 설치되지 않아** “The remote host does not meet the prerequisites for running VS Code server” 오류가 납니다.
- 따라서 PC 에는 **VS Code 1.85.2** 를 설치하고 자동 업데이트를 꺼둡니다.
 - settings.json 에 "update.mode": "none" 추가
 - 다운로드: https://code.visualstudio.com/updates/v1_85 (또는 <https://update.code.visualstudio.com/1.85.2/win32-x64-user/stable>)
- 확장 프로그램도 최신 버전이 1.85 와 호환되지 않을 수 있으므로, Remote-SSH 확장은 1.85 시절 버전으로 고정합니다(확장 우클릭 → Install Another Version).

Remote-SSH 연결

1. VS Code 에서 F1 → Remote-SSH: Connect to Host... → root@192.168.45.157 (또는 root@192.168.8.1).
2. 플랫폼은 Linux 선택, 비밀번호 oelinux123.
3. 첫 연결 시 VOXL 에 ~/.vscode-server 가 설치됩니다. 이 과정에 인터넷이 필요할 수 있으니 처음에는 **Station 모드**에서 연결하는 것을 권장합니다.

4. ~/.ssh/config 에 등록해두면 편합니다:

```
Host starling2-lab
  HostName 192.168.45.157
  User root
Host starling2-ap
  HostName 192.168.8.1
  User root
```

- 작업 폴더는 /data/ 아래에 두는 것을 권장합니다(/data 는 용량이 크고 SDK 재플래시 시 보존 옵션이 있음). ModalAI 설정 파일은 /etc/modalai/에 있습니다.

파일 전송 (SCP)

PC → 드론

```
scp my_script.py root@192.168.45.157:/data/
scp -r my_project/ root@192.168.45.157:/data/
```

드론 → PC (예: PX4 로그)

```
scp -r root@192.168.45.157:/data/px4/log ./px4_logs
```

- USB 연결 시: adb push my_script.py /data/, adb pull /data/px4/log/session001/log001.ulg .
- SDK 1.6 시스템 이미지에는 rsync 도 포함되어 있어 대용량 동기화에 쓸 수 있습니다.

nano 편집기

- 기본 이미지에는 nano 가 없어 직접 설치했습니다(패치노트 참고). Station 모드에서:

```
apt update && apt install nano
```

- 자주 편집하는 파일:
 - /etc/modalai/vox1-vision-hub.conf — offboard 모드, MAVSDK 용 localhost UDP
 - /etc/modalai/vox1-px4.conf — GPS/MAG/RC 종류(RC=CRSF_RAW 등)
 - /etc/modalai/vox1-camera-server.conf — 카메라 해상도/FPS
- 편집 후 해당 서비스 재시작: systemctl restart vox1-vision-hub 등.

조종기 사용

스위치 배치

- Starling 2 순정 조종기는 SKU 에 따라 **iFlight Commando 8 (ELRS 3.x, 915 MHz)** 또는 **Orqa (Ghost 2.4 GHz)** 입니다. ELRS 송신기 ↔ Ghost 수신기는 서로 호환되지 않습니다.
- Commando 8 은 2 단 스위치 2 개 + 3 단 스위치 2 개를 가집니다. ModalAI 공식 프리셋(voxl-px4-params/radio_helpers/Commando_8.params)의 채널 매핑은 아래와 같습니다. 우리 조종기의 실제 스위치 위치는 QGroundControl → Vehicle Setup → **Flight Modes / Radio** 페이지에서 반드시 확인하고 표를 채웁니다:

채널	기능 (공식 프리셋)	물리 스위치 위치 (우리 조종기)
ch1~4	롤 / 피치 / 스로틀 / 요	스틱
ch5	Kill switch — 해제(아래) / 킬(위): 모터 즉시 정지	[확인 필요]
ch6	비행 모드 (3 단) — 앞(위) = Manual / 중간 = Position / 뒤(아래) = Offboard	[확인 필요]
나머지	프리셋에서는 미할당(RC_MAP_* = 0) — 필요 시 QGC 에서 Altitude/Land 등 추가	[확인 필요]

- 바인딩(ELRS): 드론에서 voxl-elrs bind → 수신기 LED 더블 블링크 → 조종기 ELRS Lua 스크립트에서 [Bind]. 이후 **드론 전원을 재인가**해야 합니다. 상태 확인은 voxl-elrs ping, voxl-elrs get-link-stats, voxl-elrs get-rc.
- PX4 쪽 RC 프로토콜은 /etc/modalai/voxl-px4.conf 의 RC= 값(ELRS: CRSF_RAW, Ghost: GHST)으로 결정됩니다. 틀리면 px4-listener input_rc 가 데이터를 못 받고 시동이 안 걸립니다.

비행 모드

PX4 모드	위치추정 필요?	용도
Manual / Stabilized	불필요	자세만 안정화. VIO 상태와 무관하게 뜰 수 있어 첫 이륙에 사용
Altitude	불필요(기압계)	고도 유지, 수평은 수동
Position	필요(실내: VIO)	손을 놓으면 제자리 호버. 실내 비행의 기본 모드
Offboard	필요	MAVSDK/voxl-vision-hub 가 조종. 자율 비행용
Land	필요	자동 착륙
Return / Mission	전역 위치(GPS) 필요	실내에서는 사용 불가

- ModalAI 권장 순서(“Flying with VIO”): 이륙 지점에서 배터리 연결(VIO 원점) → **Manual** 로 이륙 → 안정되면 **Position** → 필요 시 **Offboard**.
- Position 모드로 바꾸기 전에 반드시 VIO 상태 확인: voxl-inspect-vio(vision-hub 가 정렬한 VIO, Open-VINS/QVIO 공통) 또는 voxl-inspect-vins(Open-VINS 원시 출력)에서 state OKAY, quality > 0, features 가 수십 개 이상. 구형 QVIO 를 쓰는 경우 voxl-inspect-qvio. 포털의 VIO 탭에서도 확인 가능.
- ModalAI 공식 문구: “Manual 로 이륙 후 공중에서 Position 으로 전환하는 것보다 **Position 모드로 바로 이륙**하는 것이 더 안전”합니다. VIO 가 OKAY 라면 처음부터 Position 으로 이륙해도 됩니다. VIO 가 없으면 PX4 는 자동으로 Altitude 모드로 폴백합니다.

시동/종료 절차

시동(Arm)

1. 킬 스위치 **해제(아래)** 확인, 비행 모드 스위치 Manual.
2. QGC 또는 포털 상단에서 배터리·모드·“Ready to fly” 확인.

3. 왼쪽 스틱을 **아래-오른쪽**(스로틀 최저 + 요 우측)으로 2~3 초 유지 → 모터가 아이들로 회전.
4. 회전이 이상하거나(방향/소음) 경고가 뜨면 즉시 왼쪽 스틱 **아래-왼쪽**으로 시동 해제.

종료(Disarm)

1. 착륙 후 스로틀을 최저로 유지 → 자동 해제되거나, 왼쪽 스틱 **아래-왼쪽**으로 수동 해제.
2. 모터가 완전히 멈춘 뒤 킬 스위치를 **위로** 올려 잠금.
3. 배터리 분리 전 파일 변경이 있었으면 sync.

수동 비행 팁

- 첫 비행은 프로펠러를 **뺀** 상태로 시동.모터 회전 방향을 확인합니다.
- Manual 모드에서는 스틱을 놓아도 드론이 흘러가므로 작은 입력으로 보정하며, Position 모드로 넘어가기 전 VIO 품질을 확인합니다.
- VIO 는 **질감 있고 밝은 환경**에서 잘 동작합니다. 무지 콘크리트 바닥, 흰 벽만 보이는 곳, 어두운 곳, 움직이는 풀밭은 피합니다. 트래킹 카메라 최소 초점거리는 약 15 cm 이므로 벽.바닥에 너무 붙지 않습니다.
- 이륙 직후에는 천천히 움직여 VIO 가 수렴할 시간을 줍니다. 급격한 요(yaw) 회전은 VIO 드리프트를 키웁니다.
- Offboard 테스트 중에는 항상 한 사람이 조종기를 잡고 있다가 이상 시 모드 스위치를 Position/Manual 로 내려 즉시 제어권을 회수합니다.
- 실내에서는 프로펠러 가드/네트, 보호안경을 사용하고, 천장 높이를 확인합니다(구 SDK 에서 trajectory 모드 중 천장으로 상승한 사례 있음 → SDK 1.6 이상 사용).

자율 비행

MAVSDK 서버 실행

1) **voxl-vision-hub** 설정 (/etc/modalai/voxl-vision-hub.conf)

```
"en_localhost_mavlink_udp": true,  
"localhost_udp_port_number": 14551,  
"offboard_mode": "off"
```

- offboard_mode 기본값은 figure_eight 입니다. **off** 로 바꾸지 않으면 **Offboard 모드 진입 시 vision-hub 가 스스로 8 자 비행을 시작하므로 반드시 변경합니다.**
- 저장 후 `systemctl restart voxl-vision-hub, voxl-inspect-services` 로 voxl-vision-hub, voxl-mavlink-server, voxl-px4 가 모두 실행 중인지 확인합니다.

2) 포트 정리

포트	용도
UDP 14550	voxl-mavlink-server → 외부 GCS(QGC)/외부 PC. 기본 대상 IP 192.168.8.10/11 (/etc/modalai/voxl-mavlink-server.conf)
UDP 14551	voxl-vision-hub 가 localhost 전용 으로 열어주는 MAVSDK/MAVROS 포트

3) mavsdk_server 실행 (드론 위에서)

- aarch64 용 pip 휠에는 mavsdk_server 바이너리가 포함되어 있지 않으므로 서버를 따로 실행해야 합니다.
- 우리 환경(Python 3.6.9 + mavsdk 0.12.0)은 공식 지원 조합(Python ≥ 3.7)이 아니므로, 서버 바이너리는 mavsdk 0.12.0 과 맞는 MAVSDK 릴리스(<https://github.com/mavlink/MAVSDK/releases>)에서 mavsdk_server_linux-arm64-musl 을 받아 사용합니다 [사용 중인 바이너리 버전 확인 필요].

```
cd /data/mavsdk  
chmod +x mavsdk_server_linux-arm64-musl  
./mavsdk_server_linux-arm64-musl -p 50051 udp://:14551
```

- ModalAI 공식 경로는 voxl-docker-mavsdk 컨테이너(/data/docker/mavsdk/run-docker.sh)이지만, 우리는 네이티브 Python 으로 운용합니다.

비행 코드 실행

```
import asyncio
from mavsdk import System

async def run():
    # 서버를 직접 띄웠으므로 자동 실행을 막고 주소/포트만 지정
    drone = System(mavsdk_server_address="localhost", port=50051)
    await drone.connect()

    async for state in drone.core.connection_state():
        if state.is_connected:
            print("-- Connected")
            break

    async for health in drone.telemetry.health():
        if health.is_local_position_ok: # 실내: VIO 기반 로컬 위치가 OK 여야
            break

    await drone.action.arm()
    await drone.action.takeoff()
    await asyncio.sleep(5)
    await drone.action.land()

asyncio.get_event_loop().run_until_complete(run())
```

- 실행: 다른 SSH 터미널에서 `python3 /data/scripts/takeoff_and_land.py`.
- 참고 예제: MAVSDK-Python 저장소 `examples/` (`takeoff_and_land.py`, `offboard_position_ned.py` 등). 0.12.0 API 와 최신 문서 API 가 다를 수 있으니 `pip show mavsdk` 버전에 맞는 예제를 사용합니다.
- ModalAI 는 실기 전 **HITL** 로 먼저 검증할 것을 권장합니다.

안전 절차

- 조종기는 항상 켜두고 **모드 스위치로 Offboard** 를 빠져나오면 즉시 수동 제어로 복귀한다는 점을 모든 참여자가 알고 있어야 합니다.
- 킬 스위치가 QGC 에서 매핑되어 있는지 매 세션 확인.
- PX4 안전 파라미터(QGC → Safety):

- 지오펜스: GF_ACTION, GF_MAX_HOR_DIST, GF_MAX_VER_DIST (실내 천장 높이보다 낮게)
- RC 상실: NAV_RCL_ACT, COM_RC_LOSS_T
- 저전압: COM_LOW_BAT_ACT (0 경고 / 2 착륙 / 3 귀환·착륙)
- 스크립트에는 항상 예외 처리로 land()가 호출되도록 작성하고, 첫 실행은 이륙 고도 0.5~1 m, 저속으로 제한합니다.
- 이륙 전 px4-listener vehicle_local_position 에서 X/Y/Z 가 모두 갱신되는지 확인(Z 만 움직이고 X/Y 가 0 에 고정되면 EKF2 파라미터 문제).

로그 확인

- 스크립트 로그: `python3 script.py 2>&1 | tee /data/logs/flight_$(date +%F_%H%M).log`
- vision-hub 동작 확인: `journalctl -u vox1-vision-hub -f`, 오프보드 디버그는 `vox1-vision-hub --debug_offboard`(서비스 중지 후 수동 실행).
- PX4 비행 로그(ulog): `/data/px4/log/sessNNN/logNNN.ulg` — 아래 “로그 다운로드” 참고.
- 실시간 텔레메트리: `px4-listener vehicle_local_position, px4-listener battery_status, vox1-inspect-mavlink`.

웹 대시보드 (Portal)

접속

- 브라우저에서 `http://<드론 IP>/` (포트 80). SoftAP 면 `http://192.168.8.1/`, 연구실 WiFi 면 `http://192.168.45.157/`.
- 서비스: `vox1-portal` (`systemctl status vox1-portal`, 설정은 `vox1-configure-portal`).

주요 페이지

탭	내용
Cameras	모든 이미지 파이프 실시간

탭	내용
	보기 (tracking_front/down, hires, tof 등), Multi-View 콜라주
Pointclouds	ToF/스테레오 포인트클라우드 3D 뷰
VIO	VIO 궤적 실시간 표시 — Position 모드 전 확인용
Debug	센서/기압계 캘리브레이션, RC 설정, 모터 테스트, 헬스 체크
ELRS	ELRS 수신기 상태/바인딩
Follow / Mapper / Benchmark	follow_tag 설정, vox1-mapper(설치 시), 성능 벤치

- 우측 상단에 배터리 %, 시동 상태, 비행 모드가 표시됩니다. unknown 이면 vox1-mavlink-server 또는 vox1-px4 가 실행 중이 아닙니다.

활용 팁

- 카메라 스트리밍은 CPU/발열을 크게 올리므로 **한 번에 하나의 카메라만** 보고, 특히 hires + Multi-View 동시 스트리밍은 피합니다(과열 사례 보고됨).
- Debug 탭의 모터 테스트는 **프로펠러를 뺀 상태**에서만 사용.
- VIO 탭에서 궤적이 튀거나 리셋되면 vox1-inspect-vio 로 quality/에러 비트를 확인합니다.

자주 하는 작업

발열 확인

```
vox1-inspect-cpu          # 코어별 MHz / °C / 사용률, GPU, 메모리, governor
vox1-inspect-cpu -f       # 상위 프로세스 포함
vox1-inspect-cpu -t       # 1 회 출력
```

- vox1-cpu-monitor 서비스가 켜져 있어야 합니다.

- 유희 시 약 43~44 °C 가 정상. **약 90 °C 부터 스로틀링**이 시작되고 95 °C 부근에서 클러크가 크게 떨어집니다(ModalAI 공식 답변). 비행 중에는 프로펠러 바람으로 냉각되지만, 벤치에서 오래 켜둘 때는 25×25 mm 5 V 팬을 VOXL 2 위에 올려둡니다.
- `vox1-set-cpu-mode perf|auto|powersave` 로 governor 를 바꿀 수 있습니다(perf 는 스로틀링이 더 잘 보임).

CPU/프로세스 확인

```
vox1-inspect-cpu -f
top # 또는 htop (설치되어 있으면)
vox1-inspect-services # vox1-* 서비스 실행/활성화 상태 한눈에
ps aux | grep vox1
```

로그 다운로드

```
# PX4 uLog 전체
scp -r root@192.168.45.157:/data/px4/log ./px4_logs
# 특정 파일
adb pull /data/px4/log/sess001/log001.ulg .
```

- ulog 는 기본적으로 시동~해제 구간만 기록됩니다. 부팅부터 기록하려면 `/etc/modalai/vox1-px4.conf` 의 logger 옵션에 `-e` 를 추가하고, 전원을 끄기 전 `px4-logger stop` 을 실행합니다.
- 분석: <https://logs.px4.io> (Flight Review)에 업로드, 또는 QGC → Analyze → Log Download.
- MPA 파이프(VIO, IMU, 카메라) 기록: `vox1-logger --preset_odometry --time 60 --note "vio test" → /data/vox1-logger/`에 저장, `vox1-replay` 로 재생. 시동 중에만 자동 기록하려면 `systemctl enable --now vox1-logger-auto`.

파라미터 백업

- PX4 파라미터는 기본값과의 **차이만** `/data/px4/param/parameters` 파일에 저장됩니다(이 파일을 지우면 공장 기본값으로 복귀).

```
# 백업
scp root@192.168.45.157:/data/px4/param/parameters ./px4_params_$(date +%F)
# 현재 값 확인
px4-param show
```

- QGC → Vehicle Setup → Parameters → Tools → **Save to file** 로도 백업/복원 가능.
- 프리셋 적용: `vox1-configure-px4-params -w` (위저드) → EKF2_helpers → `indoor_vio_missing_gps.params` (실내 VIO 전용). 프리셋 원본: <https://gitlab.com/vox1-public/vox1-sdk/utilities/vox1-px4-params>
- 캘리브레이션 파일도 함께 백업: `/data/modalai/vox1-imu-server.cal`, `/data/modalai/opencv_tracking*_intrinsics.yml`, `/data/px4/param/parameters_{gyro,acc,mag,level}.cal`. 상태 확인은 `vox1-check-calibration`.

트러블슈팅

시동 안 걸릴 때

먼저 QGC 상단 메시지가 `px4-commander check` 로 **거부 사유**를 확인합니다.

메시지/증상	원인	조치
“No valid local position estimate” / “vision position estimate not available”	VIO 가 EKF2 에 융합되지 않음	아래 “EKF 관련 에러” 참고
“Compass sensor 0 missing”	실내 세트에 자력계 없음	<code>SYS_HAS_MAG 0</code> , <code>EKF2_MAG_TYPE 5</code> , <code>SENS_MAG_MODE 0</code>
RC 신호 없음 (<code>px4-listener input_rc</code> 비어 있음)	바인딩 안 됨 또는 <code>vox1-px4.conf</code> 의 <code>RC=</code> 값 오류	ELRS 는 <code>RC=CRSF_RAW</code> , 재바인딩 후 전원 재인가
킬 스위치 켜짐	스위치 위치	킬 스위치 아래로
배터리 저전압	방전	충전 후 재시도
RC 미캘리브레이션	새 조종기	QGC → Radio → Calibrate

- 실내 VIO 비행용 권장 파라미터(포럼에서 Starling 2 계열로 검증): `EKF2_EV_CTRL 15`, `EKF2_HGT_REF 3`, `COM_ARM_WO_GPS 1`, `SYS_HAS_GPS 0`, `SYS_HAS_MAG 0`, `EKF2_MAG_TYPE 5`, `SENS_MAG_MODE 0`, `vox1-px4.conf` 에 `GPS=NONE`, 그리고 `indoor_vio_missing_gps.params` 프리셋 적용.

EKF 관련 에러

1. VIO 상태 확인: `vox1-inspect-vio` (Open-VINS 원시 출력은 `vox1-inspect-vins`,
구형 QVIO 는 `vox1-inspect-qvio`)
 - state: FAIL(0) / INIT(1) / OKAY(2)
 - quality: -1 이면 실패, 값이 클수록 좋음 (Open-VINS 기본 임계값: 14 이하 나쁨, 35 이상 좋음)
 - 에러 비트: 0x1 COV_ERROR, 0x10 NO_FEATURES, 0x800 BAD_CAM_CAL, (QVIO) IMU_OOB, IMU_BW, NOT_STATIONARY
2. Open-VINS 는 움직이는 중에도 수십 ms 내 초기화됩니다. 구형 QVIO 는 **정지 상태**에서만 초기화되며 성공할 때까지 3 초마다 리셋하므로 드론을 바닥에 가만히 두고 기다립니다.
3. 리셋/재시작:

```
vox1-reset-vins # Open-VINS 리셋 (QVIO 는 vox1-reset-qvio)
systemctl restart vox1-open-vins-server # (QVIO 는 vox1-qvio-server)
systemctl restart vox1-vision-hub
vox1-inspect-services # VIO 서비스가 하나만 켜져 있는지 확인 (둘 다 켜면 실패)
```

4. `px4-listener vehicle_local_position` 에서 **z** 는 바뀌는데 **x/y** 가 **0** 에 고정되면 VIO 가 아니라 EKF2 융합 파라미터(EKF2_EV_CTRL 등) 문제입니다.
5. 환경 개선: 조명 확보, 바닥/벽에 질감(포스터, 테이프 패턴) 추가, 카메라 렌즈 청소.
6. 그래도 안 되면 전원 재인가 후 1)부터 반복.

VSCode 연결 실패

증상	조치
"The remote host does not meet the prerequisites for running VS Code server"	glibc 문제. VS Code 1.85.x 사용, "update.mode": "none" 확인. 1.86+로 자동 업데이트되었으면 재설치
서버 설치 중 멈춤	드론이 인터넷에 연결(Station 모드)되어 있는지 확인, /data 여유 공간 확인(df -h)

증상	조치
호스트를 못 찾음	/data) IP 변경 여부(voxl-my-ip), 같은 네트워크인지, 터미널에서 ssh root@<ip>가 되는지 먼저 확인
확장 프로그램 오류	Remote-SSH 확장을 1.85 호환 버전으로 다운그레이드 대안: VS Code 1.85 대신 오픈소스 “Open Remote - SSH” 확장(포럼 사용자 성공 사례) 또는 scp + nano 로 작업.

발열 문제

- 증상: voxl-inspect-cpu 에서 클럭(MHz)이 계속 떨어짐, 카메라 프레임 드롭, VIO 품질 저하, 포털 지연.
- 원인: 벤치에서 장시간 전원 인가 + 카메라 스트리밍(특히 hires) + 직사광/무풍.
- 조치: 팬으로 송풍, 포털 스트리밍 중지, 불필요한 서비스 중지(systemctl stop voxl-tflite-server 등), 비행 사이에 전원 끄기. 90 °C 이상이 지속되면 비행하지 않습니다.

배터리 관련

- 상태 확인: voxl-inspect-battery (전압/잔량/전류/전력, vision-hub 와 PX4 가 실행 중이어야 함), 포털 우측 상단.
- 저전압 failsafe 파라미터: BAT_LOW_THR(경고, 예: 0.15), BAT_CRIT_THR, BAT_EMERGEN_THR, COM_LOW_BAT_ACT. 포럼에서 Starling 2 용으로 BAT1_V_EMPTY 2.8~2.9 가 사용된 사례 있음 [우리 값 확인 필요].
- QGC 에서 배터리가 200 V 대로 표시되는 것은 구 voxl-px4 의 voltages_ext 버그 → SDK 업데이트로 해결.
- 3S LiPo 사용 금지** (모터/ESC 2S 사양). 순정 2S Li-Ion 또는 XT30 커넥터의 2S 18650 팩만 사용.

안전 수칙

비행 전 체크리스트

- ☐ 배터리 완충(4.2V/셀) 및 벨크로/스트랩 고정
- ☐ 프로펠러 체결·회전 방향 확인, 손상 없음 (첫 세션은 프로펠러 제거 후 시동 테스트)
- ☐ 커넥터·안테나·케이블 이상 없음, 카메라 렌즈 청결
- ☐ 조종기 바인딩 확인, **킬 스위치 매핑을 QGC 에서 확인**
- ☐ 드론을 들어 기울였을 때 QGC 자세 표시가 따라 움직임
- ☐ `vox1-inspect-vio`: state OKAY, quality > 0 (또는 포털 VIO 탭)
- ☐ `vox1-inspect-services`: `vox1-px4`, `vox1-vision-hub`, `vox1-mavlink-server`, `vox1-camera-server` 정상
- ☐ `vox1-inspect-cpu`: 온도 정상
- ☐ 야외라면 GPS 락, QGC 프리플라이트 전부 녹색
- ☐ 이륙 지점 = VIO 초기화 지점(배터리 연결 위치)
- ☐ 주변 인원 이격, 보호안경, 실내 네트/가드, 천장 높이 파악
- ☐ Offboard 테스트라면 `offboard_mode`: off 및 조종기 담당자 지정

비상 대응

1. **제어 이상/스크립트 폭주**: 모드 스위치를 Position 또는 Manual 로 → 수동 제어권 회수.
2. **충돌 임박/인명 위험**: **킬 스위치 위** → 모터 즉시 정지(드론이 추락하므로 저고도·네트 위에서만 사용 판단).
3. **VIO 상실로 드리프트**: Manual/Altitude 로 전환해 수동 착륙. Position 모드 유지 금지.
4. **RC 상실**: `NAV_RCL_ACT` failsafe(착륙 권장)에 맡기되, 사전에 설정 확인.
5. **배터리 경고**: 즉시 착륙. `COM_LOW_BAT_ACT` 를 2(착륙)로 두면 자동 착륙.
6. 착륙 후 스로틀 최저 유지 → 시동 해제 → 킬 스위치 위 → 배터리 분리.

- 충돌 후에는 프로펠러.프레임.카메라 마운트 점검, IMU 재캘리브레이션(voxl-calibrate-imu) 고려.

배터리 관리

- 종류: **2S Li-Ion 18650** (LiPo 아님). 최대 4.2 V/셀, 충전은 **1C 이하(약 3 A)** 로 Li-Ion 모드가 있는 밸런스 충전기 사용.
- 충전 시 드론에서 배터리를 분리합니다**(ModalAI: 충전 중 사용은 어떤 하드웨어도 검증되지 않음). USB-C 로는 충전되지 않습니다.
- 보관: 장기 보관 시 셀당 약 3.6~3.8 V(일반 Li-Ion 관례, ModalAI 공식 수치 없음). 비행 중 셀당 약 3.3 V 이하로 방전시키지 않습니다.
- 충전은 내화 백/안전한 장소에서, 충돌 후에는 외관.전압 점검 후 사용. 부풀거나 손상된 팩은 폐기.
- 순정 교체팩: ModalAI M10000538 (4000 mAh 2S1P, 7.4 V, XT30, 108 g).

패치노트

nano 설치 완료

- 기본 이미지에 없던 nano 를 Station 모드에서 `apt install nano` 로 설치. 설정 파일 편집용.

파이썬 3.6.9 환경

- Ubuntu 18.04 기본 Python 3.6.9 사용(시스템 이미지 고정). pip3 는 `apt install python3-pip` 로 설치.
- 주의: voxl-* 패키지를 `apt upgrade` 로 개별 업데이트하는 것은 지원되지 않음(SDK 전체 플래시 방식). Python 패키지만 pip 로 관리.

mavsdk 0.12.0

- Python 3.6 에서 설치 가능한 MAVSDK-Python 버전으로 `mavsdk==0.12.0` 사용. aarch64 휠에는 `mavsdk_server` 가 없어 서버 바이너리를 별도로 실행(위 “MAVSDK 서버 실행” 참고).

vscode 설치 1.85.2

- PC 에 VS Code 1.85.2 고정 설치, 자동 업데이트 비활성화("update.mode": "none"). Ubuntu 18.04 glibc 2.27 호환 마지막 계열.

참고 자료

기술 문서 링크

- 내부 문서: 「Starling 2 기술 문서」 (같은 폴더) — 시스템 구조, VIO/EKF2, 카메라 파이프, MPA, 서비스, 캘리브레이션 상세

공식 docs 링크

- Starling 2: <https://docs.modalai.com/starling-2/> · 데이터시트: <https://docs.modalai.com/starling-2-datasheet/> · 구조: <https://docs.modalai.com/voxl2-d0014/>
- 개발자 부트캠프: <https://docs.modalai.com/voxl-developer-bootcamp/>
- VOXL SDK / voxl-suite: <https://docs.modalai.com/voxl-sdk/> , <https://docs.modalai.com/voxl-suite/> · 다운로드: <https://downloads.modalai.com>
- WiFi: <https://docs.modalai.com/voxl-2-wifi-setup/> · ADB: <https://docs.modalai.com/setting-up-adb/> · 버전 확인: <https://docs.modalai.com/voxl-version/>
- PX4 on VOXL: <https://docs.modalai.com/voxl-px4/> · 파라미터 프리셋: <https://gitlab.com/voxl-public/voxl-sdk/utilities/voxl-px4-params> · VIO 비행: <https://docs.modalai.com/flying-with-vio/>
- RC/ELRS: <https://docs.modalai.com/voxl2-rc-configs/> , <https://docs.modalai.com/voxl-elrs/> · 첫 비행: <https://docs.modalai.com/fc-first-flight/>
- MAVSDK: <https://docs.modalai.com/mavsdk/> , <https://github.com/mavlink/MAVSDK-Python> · voxl-vision-hub: <https://docs.modalai.com/voxl-vision-hub/> · MAVLink 라우팅: <https://docs.modalai.com/mavlink/>
- Portal: <https://docs.modalai.com/voxl-portal/> · Logger: <https://docs.modalai.com/voxl-logger/> · Inspect 도구:

<https://docs.modalai.com/inspect-tools/> · 발열: <https://docs.modalai.com/voxl2-thermal-performance/>

- PX4 문서: https://docs.px4.io/main/en/complete_vehicles_mc/modalai_starling , 비행 모드 https://docs.px4.io/main/en/flight_modes_mc/ , 안전 <https://docs.px4.io/main/en/config/safety.html> , Flight Review <https://logs.px4.io>
- ModalAI 포럼(Starling 카테고리): <https://forum.modalai.com/category/44/starling-starling-2>